**Materia:** Programación IV

**UNIDAD 7** · SERVER STATE Y NAVEGACIÓN

APUNTE TEÓRICO

# Optimización de Endpoints en FastAPI

**Revisor de la unidad:** Prof. Franco González

*Implementación de paginación, filtros dinámicos y respuestas estandarizadas*

## 1. Introducción

El desarrollo de APIs modernas requiere algo más que la simple implementación de operaciones CRUD (Create, Read, Update, Delete). A medida que las aplicaciones crecen y comienzan a manejar grandes volúmenes de datos, se vuelve necesario implementar mecanismos que permitan consultar información de manera eficiente, segura y escalable.

En este contexto, **FastAPI** es un framework ampliamente adoptado para el desarrollo de APIs en Python. Su popularidad se debe principalmente a su alto rendimiento, su integración con tipado estático y su sistema de validación automática basado en Pydantic. Estas características permiten construir servicios web robustos con una estructura clara y mantenible.

Sin embargo, una API que simplemente devuelve todos los registros de una base de datos puede generar múltiples problemas cuando el volumen de datos aumenta. Entre estos problemas se encuentran el consumo excesivo de memoria, tiempos de respuesta elevados y dificultades para que el frontend procese correctamente la información recibida.

Para resolver estas limitaciones se utilizan diversas estrategias de optimización. Entre las más importantes se encuentran:

- **Paginación de resultados**
- **Filtros dinámicos**
- **Validación avanzada de parámetros**
- **Respuestas estandarizadas**

Estas técnicas permiten transformar un endpoint básico en un **endpoint profesional preparado para entornos de producción**.

El objetivo de este documento es analizar cómo implementar estas estrategias utilizando **FastAPI y SQLModel**, tomando como caso de estudio el desarrollo de un endpoint de listado de armas (`/weapons`). Este ejemplo permite ilustrar cómo integrar consultas eficientes a la base de datos,

validación automática de parámetros y estructuras de respuesta adecuadas para aplicaciones frontend.

## 2. Problemas de un endpoint de listado básico

En las primeras etapas del desarrollo de una API es común implementar endpoints de forma simple. Un ejemplo típico es un endpoint que consulta todos los registros de una tabla y los devuelve al cliente.

Un ejemplo conceptual podría ser:

```
GET /weapons
```

Este endpoint devuelve una lista completa de armas almacenadas en la base de datos.

Aunque este enfoque funciona correctamente en aplicaciones pequeñas, presenta varios problemas cuando el sistema comienza a crecer.

### 2.1 Problemas de rendimiento

Cuando una base de datos contiene miles o millones de registros, devolver todos los datos en una única respuesta puede generar:

- alto consumo de memoria en el servidor
- respuestas demasiado grandes
- aumento del tiempo de procesamiento
- saturación de la red

Esto impacta negativamente tanto en el backend como en el cliente que consume la API.

### 2.2 Problemas para el frontend

Otro problema importante aparece en el lado del frontend. Si el backend devuelve simplemente una lista de objetos JSON, el cliente no dispone de información suficiente para construir una interfaz de navegación adecuada.

Por ejemplo:

```
[
{ "id": 1, "name": "Excalibur" },
{ "id": 2, "name": "Mjolnir" }
]
```

Con esta estructura el frontend **no sabe**:

- cuántos registros existen en total
- cuántas páginas de resultados hay
- si la página actual es la última

Esto dificulta la implementación de interfaces con paginación.

### 2.3 Falta de control sobre las consultas

Si los parámetros de consulta no están correctamente validados, los clientes podrían enviar peticiones problemáticas como:

```
GET /weapons?limit=100000
```

o

```
GET /weapons?offset=-200
```

Este tipo de consultas pueden generar:

- sobrecarga en la base de datos
- consultas innecesariamente costosas
- errores en la aplicación

Por estas razones es fundamental implementar mecanismos de control y optimización.

## 3. Paginación de resultados

La **paginación** es una técnica utilizada para dividir grandes conjuntos de datos en partes más pequeñas llamadas páginas.

En lugar de devolver todos los registros en una sola respuesta, el servidor devuelve únicamente una porción de los datos.

### 3.1 Concepto de offset y limit

El patrón más utilizado en bases de datos relacionales es el uso de los parámetros:

- `limit`
- `offset`

`limit`

Define la cantidad máxima de registros que se devuelven en la respuesta.

`offset`

Indica cuántos registros iniciales deben omitirse antes de comenzar a devolver resultados.

Por ejemplo:

```
SELECT * FROM weapon
ORDER BY id
LIMIT 10 OFFSET 20;
```

Esto significa:

- omitir los primeros 20 registros
- devolver los siguientes 10

### 3.2 Beneficios de la paginación

La paginación ofrece múltiples ventajas:

- mejora el rendimiento de la API
- reduce el consumo de memoria
- disminuye el tamaño de las respuestas
- permite construir interfaces de navegación en el frontend

## 4. Implementación de paginación con SQLModel

SQLModel es una biblioteca que combina características de **SQLAlchemy** y **Pydantic** y permite definir modelos de base de datos mediante anotaciones de tipos.

Para implementar paginación se utilizan los métodos:

- `offset()`
- `limit()`

Un ejemplo conceptual sería:

```
query = select(Weapon).order_by(Weapon.id)

query = query.offset(offset)
query = query.limit(limit)

results = session.exec(query).all()
```

En este caso:

- `offset` indica cuántos registros saltar
- `limit` define cuántos registros devolver

Esto garantiza que la base de datos solo devuelva los registros necesarios para la página actual. La consulta incluye un orden definido (en este caso, por id) porque, sin una cláusula `ORDER BY`, PostgreSQL no garantiza el orden en que devuelve las filas y, entre una petición y otra, las páginas pueden repetir u omitir registros.

## 5. Filtros dinámicos

En la mayoría de las aplicaciones los usuarios necesitan buscar información utilizando diferentes criterios.

Por ejemplo:

```
GET /weapons?name=Excalibur
```

o

```
GET /weapons?description=fuego
```

Para implementar este comportamiento se utilizan **filtros dinámicos**.

### 5.1 Construcción dinámica de consultas

La estrategia consiste en:

1. crear una consulta base
2. agregar condiciones solo si existen parámetros de filtro

Ejemplo conceptual:

```
query = select(Weapon)

if name:
   query = query.where(Weapon.name.ilike(f"%{name}%"))

if description:
   query = query.where(Weapon.description.ilike(f"%{description}%"))
```

Esto permite construir consultas flexibles sin duplicar código. El operador `ilike` compara sin distinguir mayúsculas de minúsculas, y los comodines `%` que rodean al texto permiten una búsqueda parcial: con `name=espada` se obtienen todas las armas cuyo nombre contiene ese texto, mientras que una comparación con `==` solo devuelve coincidencias exactas. El filtro por `description` sigue la misma forma y busca el texto dentro de la descripción de cada arma; si el parámetro no se envía, la condición no se agrega a la consulta.

### 5.2 Ventajas de los filtros dinámicos

Los filtros dinámicos permiten:

- crear APIs más flexibles
- evitar múltiples endpoints redundantes
- mejorar la experiencia del usuario
- reducir la cantidad de datos transferidos

## 6. Validación de parámetros con Query

FastAPI proporciona un sistema avanzado de validación de parámetros mediante la función `Query`.

Esto permite establecer restricciones directamente en la firma de la función.

Ejemplo:

```
limit: int = Query(10, ge=1, le=100)
offset: int = Query(0, ge=0)
```

En este caso:

- `ge` (greater than or equal) significa **mayor o igual que**
- `le` (less than or equal) significa **menor o igual que**

Esto garantiza que:

- el límite mínimo sea 1
- el límite máximo sea 100
- el `offset` no pueda ser negativo

### 6.1 Protección de la base de datos

Estas validaciones no solo sirven para verificar los datos de entrada.

También funcionan como un mecanismo de protección contra consultas costosas o mal formadas.

Si un cliente intenta enviar una petición inválida, FastAPI responde automáticamente con un error **422 Unprocessable Content** (denominado Unprocessable Entity en especificaciones anteriores). El cuerpo de la respuesta es un JSON que indica el parámetro inválido y la regla incumplida; por ejemplo, con `offset=-1` el mensaje es "Input should be greater than or equal to 0".

# 7. Respuestas estandarizadas

Un endpoint profesional no debería devolver listas simples.

En su lugar, es recomendable devolver un objeto JSON que incluya metadatos sobre la consulta.

Ejemplo de respuesta estandarizada:

```
{
 "total": 1250,
 "items": [
   { "id": 1, "name": "Excalibur" },
   { "id": 2, "name": "Mjolnir" }
 ]
}
```

Esta estructura proporciona información adicional que el frontend puede utilizar para construir interfaces de navegación.

Para obtener el valor de `total`, el endpoint ejecuta dos consultas: primero cuenta los registros que cumplen los filtros y luego obtiene la página solicitada con `offset` y `limit`. El conteo se calcula sobre la consulta ya filtrada, de modo que `total` refleja la cantidad de resultados de la búsqueda y no la de toda la tabla:

```
count_query = select(func.count()).select_from(query.subquery())
total = session.exec(count_query).one()

items = session.exec(query.order_by(Weapon.id).offset(offset).limit(limit)).all()
```

La función `func` se importa desde `sqlmodel`, junto con `select`.

## 7.1 Información incluida

La respuesta incluye:

- **`total`** → cantidad de registros que cumplen los filtros (no la de toda la tabla)
- **`items`** → lista de resultados de la página actual

Con esta información el frontend puede calcular:

```
total_pages = ceil(total / limit)
```

La división se redondea hacia arriba (`ceil`) porque la última página puede contener menos registros que `limit`: con `total = 1250` y `limit = 10` se obtienen 125 páginas, y con `total = 1251`, 126. En JavaScript el redondeo se realiza con `Math.ceil` y en Python, con `math.ceil`. La

página actual se deduce de los parámetros que envía el propio cliente: es el cociente entero entre `offset` y `limit`, más uno (por ejemplo, con `offset = 20` y `limit = 10`, la página actual es la 3).

## 8. Modelo de respuesta paginada

Para implementar este patrón se define un modelo de respuesta utilizando SQLModel.

Ejemplo conceptual:

```
class WeaponPaginatedResponse(SQLModel):
       total: int
       items: List[WeaponRead]
```

Este modelo actúa como un contenedor que envuelve los resultados.

Luego se declara en el decorador del endpoint mediante el parámetro `response_model`:

```
@router.get("/", response_model=WeaponPaginatedResponse)
```

FastAPI valida automáticamente que la respuesta generada por el servidor cumpla con la estructura definida y serializa únicamente los campos declarados en el modelo.

El siguiente ejemplo, basado en el código del curso, integra las técnicas anteriores. El servicio construye la consulta, aplica el filtro, cuenta los resultados y devuelve la página solicitada:

```
def get_weapons(
    session: Session,
    offset: int,
    limit: int,
    name: Optional[str] = None,
) -> Tuple[int, List[Weapon]]:
    query = select(Weapon)

    if name:
        query = query.where(Weapon.name.ilike(f"%{name}%"))

    count_query = select(func.count()).select_from(query.subquery())
    total = session.exec(count_query).one()

    results = session.exec(
        query.order_by(Weapon.id).offset(offset).limit(limit)
    ).all()

    return total, list(results)
```

El router recibe y valida los parámetros, invoca al servicio y arma el objeto de respuesta:

```python
@router.get("/", response_model=WeaponPaginatedResponse)
def list_weapons(
    offset: int = Query(0, ge=0),
    limit: int = Query(10, ge=1, le=100),
    name: Optional[str] = None,
    session: Session = Depends(get_session),
):
    total, items = service.get_weapons(
        session=session,
        offset=offset,
        limit=limit,
        name=name,
    )
    return {"total": total, "items": items}
```

El servicio devuelve una tupla con el total y la lista de la página; el router arma el diccionario de respuesta y FastAPI lo valida contra `WeaponPaginatedResponse`. Las importaciones son las del código del curso: `Session`, `select` y `func` desde `sqlmodel`; `Query` y `Depends` desde `fastapi`; `Optional`, `Tuple` y `List` desde `typing`. El filtro y la paginación se combinan en la misma petición, por ejemplo: `GET /weapons?name=espada&offset=0&limit=1`. Al igual que el código del curso, este ejemplo implementa solo el filtro por `name`; el filtro por `description` se agrega con el mismo patrón: un parámetro `Optional[str]` en el router y en el servicio, y el mismo `if` sobre la consulta.

## 9. Beneficios del enfoque

La combinación de paginación, filtros dinámicos y validación ofrece múltiples beneficios.

### 9.1 Mejora del rendimiento

La base de datos solo devuelve los registros necesarios.

### 9.2 Reducción del tráfico de red

Las respuestas contienen únicamente los datos requeridos.

### 9.3 Mejor integración con el frontend

Las respuestas estandarizadas facilitan la implementación de interfaces de usuario.

### 9.4 Escalabilidad

El sistema responde con un costo acotado por página aun cuando la tabla crece. Con desplazamientos muy grandes, el `OFFSET` pierde eficiencia, porque el motor igual procesa las filas omitidas; en esos casos se recurre a otras estrategias, como la paginación por cursor.

## 10. Conclusión

La optimización de endpoints de listado es una práctica fundamental en el desarrollo de APIs modernas. A medida que los sistemas crecen y comienzan a manejar grandes volúmenes de datos, se vuelve imprescindible aplicar técnicas que permitan consultar la información de manera eficiente y segura.

En este documento se analizan cuatro estrategias principales:

1. paginación mediante `offset` y `limit`
2. filtros dinámicos para consultas flexibles
3. validación avanzada de parámetros utilizando `Query`
4. respuestas estandarizadas mediante modelos de respuesta

Aplicadas en conjunto, estas técnicas acotan el volumen de cada respuesta, rechazan parámetros inválidos antes de consultar la base de datos y entregan al cliente los datos necesarios para navegar entre páginas.

FastAPI y SQLModel proporcionan herramientas que facilitan la implementación de estas prácticas mediante tipado estático, validación automática y una integración sencilla con bases de datos relacionales.

El uso adecuado de estas estrategias mejora el rendimiento del sistema, protege la base de datos frente a consultas costosas y facilita el trabajo de los desarrolladores frontend.

En el frontend, la respuesta con `total` e `items` es la que se consume como server state mediante TanStack Query para paginar el listado. Los valores de `offset`, `limit` y los filtros forman parte de la clave de la consulta, de modo que cada página y cada búsqueda se almacenan en caché por separado. El campo `total` permite calcular la cantidad de páginas y habilitar o deshabilitar los controles de navegación.

## Bibliografía

1. FastAPI. (s. f.). *FastAPI documentation*. https://fastapi.tiangolo.com
2. PostgreSQL Global Development Group. (s. f.). *PostgreSQL documentation*. https://www.postgresql.org/docs
3. Pydantic. (s. f.). *Pydantic documentation*. https://docs.pydantic.dev
4. SQLModel. (s. f.). *SQLModel documentation*. https://sqlmodel.tiangolo.com